LANDIQ

VISUAL REFERENCE • MASTER BUILD PROMPT

Recreate the original SIH 2026 frontend experience and connect it to a fault-tolerant multimodal AI backend.

Target journey
Overview → High-Risk Cases → Acquisition Case Deep Dive → Risk Analysis & SHAP → Recommended Actions → GIS Parcel Map → Executive Briefing

Reference style
Dark navy left sidebar • white/very-light-gray workspace • blue active navigation • compact status chips • crisp cards • charts and tables • parcel-level interactive map • clear “Illustrative Data” labels

This version incorporates the seven screenshots supplied with the request as visual reference pages. It is intended to be both a product specification and a copy-ready prompt for a coding agent. The screenshots are the authority for the desired look and flow; the actual repository must still be inspected before code changes.

1. What must remain recognizable

LANDIQ is a predictive land-acquisition delay prevention platform for linear infrastructure corridors. Recreate the supplied original experience rather than designing a new generic dashboard. Maintain the same navigation, information density, sample corridor context, risk-led workflow, and decision-support storytelling.

Visual non-negotiables

- A fixed dark navy sidebar with LANDIQ branding at the top, the “PLATFORM NAVIGATION” label, icon + text links, blue selected item, and lower System Status / Prototype Mode panels.
- A slim dark top strip with “LANDIQ • PROTOTYPE”, product tagline, “Illustrative Data” badge, and ML engine label.
- A white context bar below it with corridor selector (Delhi-Mumbai Express Corridor, Package 4), corridor code, total case count, current section label, and right-aligned Case ID search.
- Main content on a pale gray background, mostly white cards with subtle blue-gray borders, rounded corners, restrained shadows, and generous section spacing.
- Use navy, cobalt blue, teal/green, amber, and red/pink semantically: blue navigation/action, green low-risk/success, amber medium/pending, red high-risk/urgent.
- Typography should resemble the supplied screenshots: bold dark headings, compact uppercase labels, readable muted secondary text, and monospace styling for IDs/technical values.
- Desktop-first 16:9 layout; avoid excessive blank space or oversized decorative graphics. Make all screens responsive without losing dashboard density.

Existing frontend stack to preserve

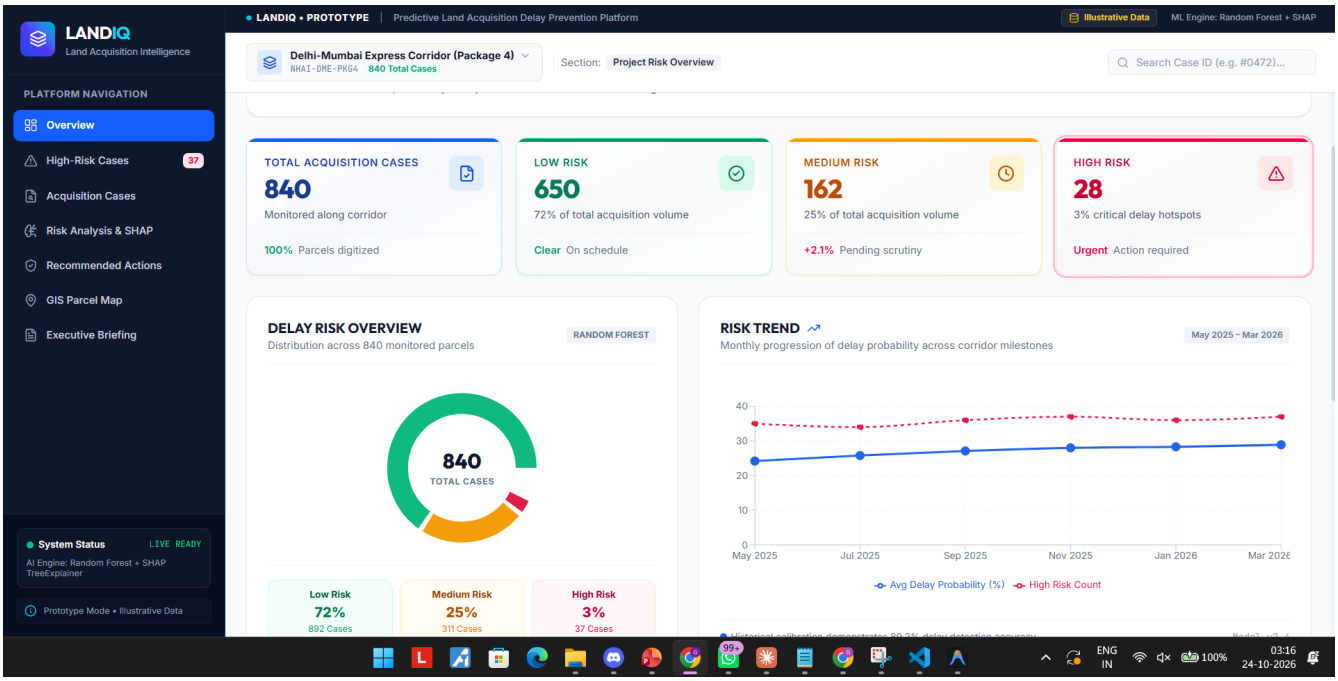
The provided project configuration identifies React + TypeScript + Vite, Tailwind CSS v4, Leaflet, Recharts, and lucide-react. Keep package.json and lockfile intact unless a verified need requires a change. Use the existing stack where possible.

Screen inventory

Navigation item	Required page behavior
Overview	Four KPI cards, risk distribution donut, monthly risk trend, corridor status, alert/summary sections.
High-Risk Cases	Searchable/filterable ranked case table with probability bars, risk chip, legal/compensation/approval/R&R;/progress columns.
Acquisition Cases	Case hero/deep dive with location, survey numbers, area/district, risk banner and five status cards.
Risk Analysis & SHAP	Explainable AI banner, baseline-to-output visual, feature contribution cards/waterfall, model context and evidence.
Recommended Actions	What-if mitigation panel, prioritized action cards, reasons, suggested authority, workflow buttons and simulation.
GIS Parcel Map	Map, parcel polygons, risk markers, hotspot focus buttons, layer toggles, legend and selected-parcel inspector.
Executive Briefing	Print/export executive dossier, model/baseline summary, top risk drivers, corridor risk summary and caveats.

REFERENCE 1 — Overview dashboard

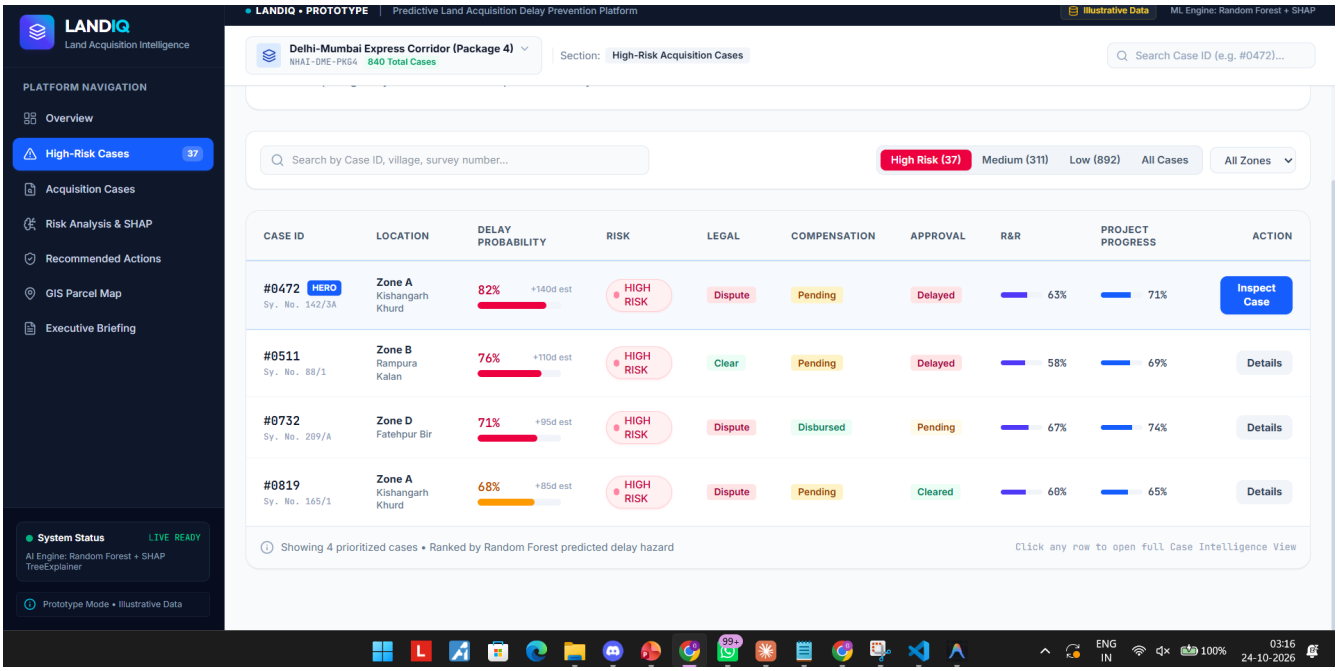
KPI cards: total cases, low risk, medium risk, high risk; risk distribution donut; risk trend chart.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 2 — High-risk case queue

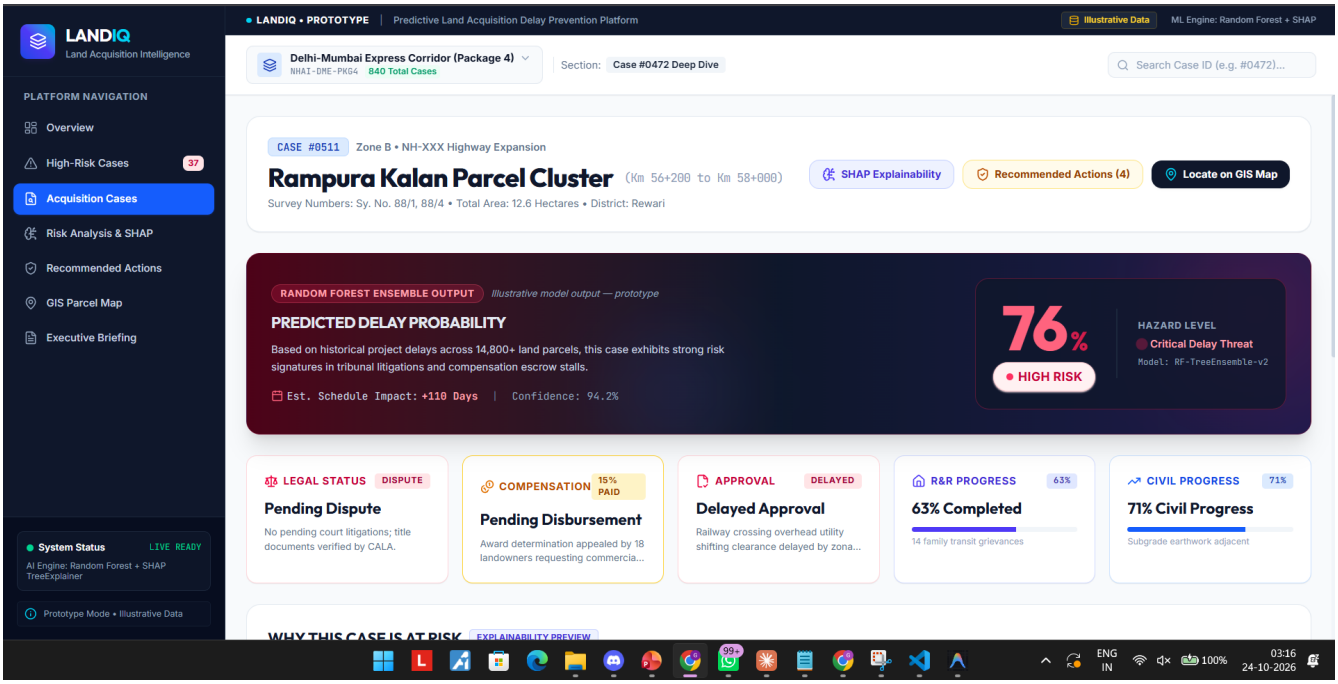
Filters for risk and zone; case table with #0472 highlighted as the hero case; probability bars and statutory status columns.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 3 — Acquisition case deep dive

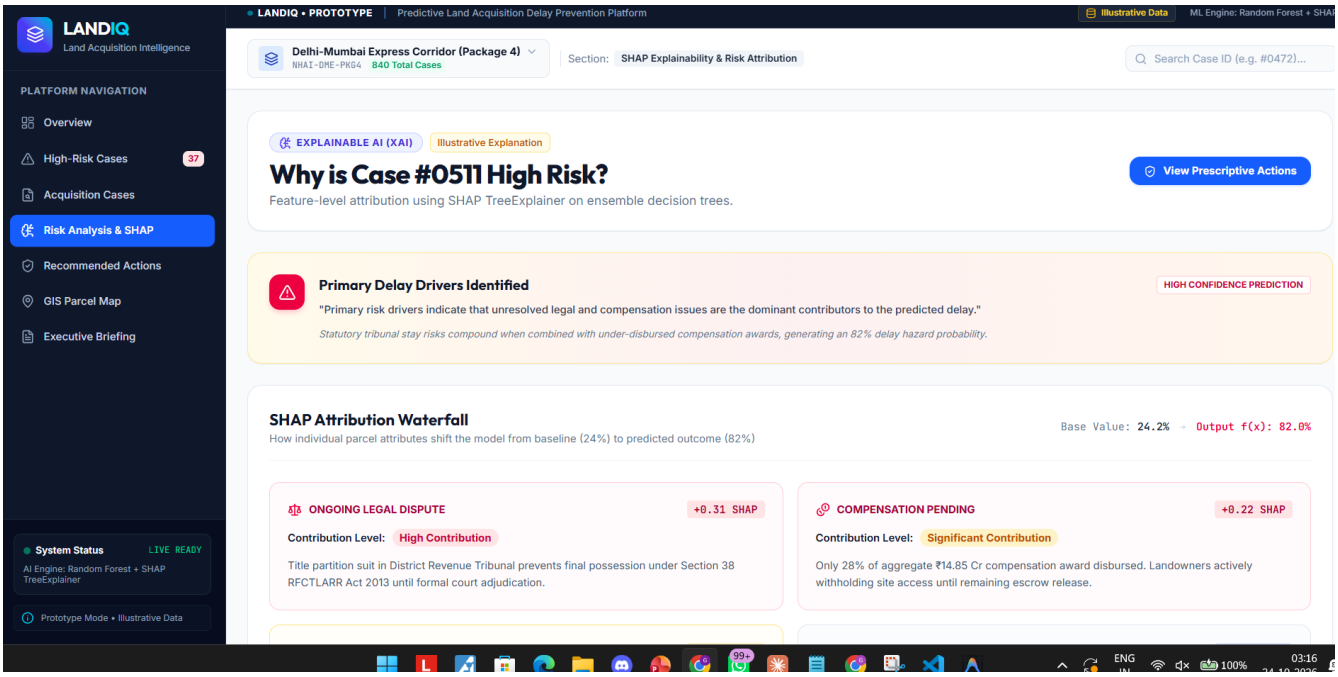
Case hero, delay-risk panel, estimated schedule impact, hazard label, and Legal / Compensation / Approval / R&R; / Civil Progress cards.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 4 — SHAP explainability

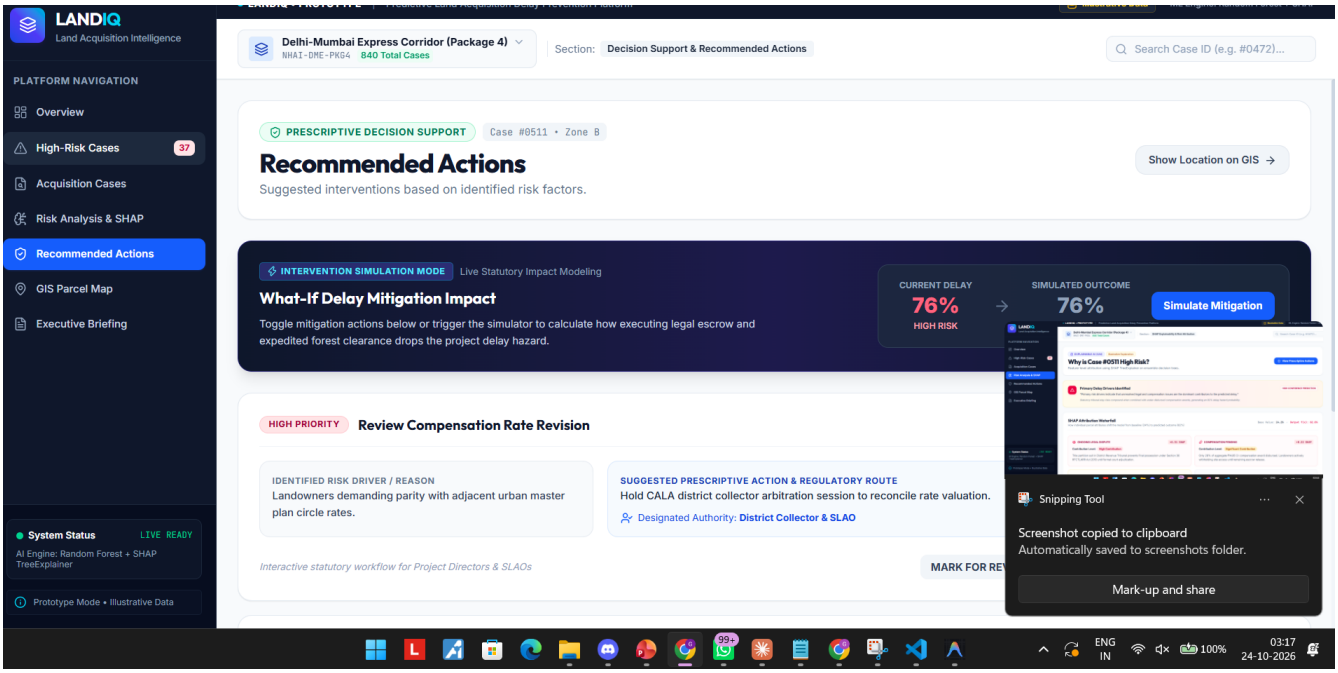
Primary delay drivers, baseline-to-output attribution, feature-level explanations and route to prescriptive actions.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 5 — Recommended actions

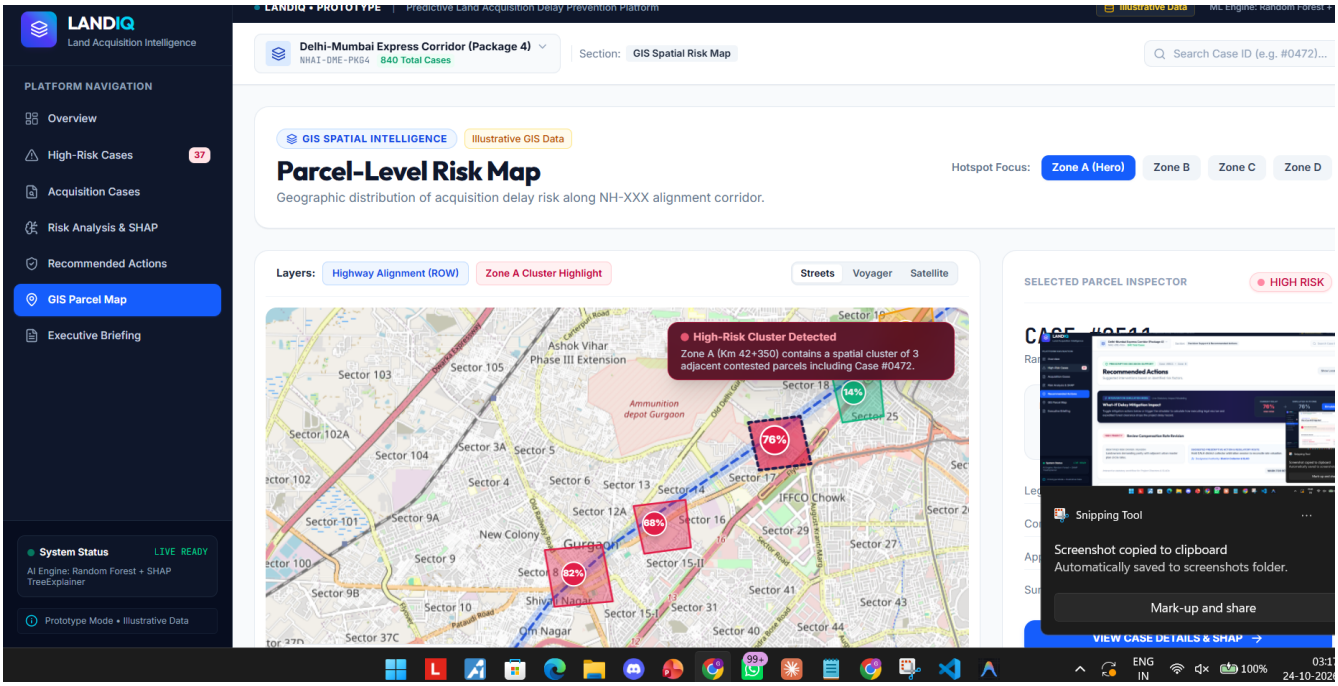
Intervention simulation panel and actionable review/escalation cards; controls should update state and show simulated outcomes honestly.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 6 — GIS parcel map

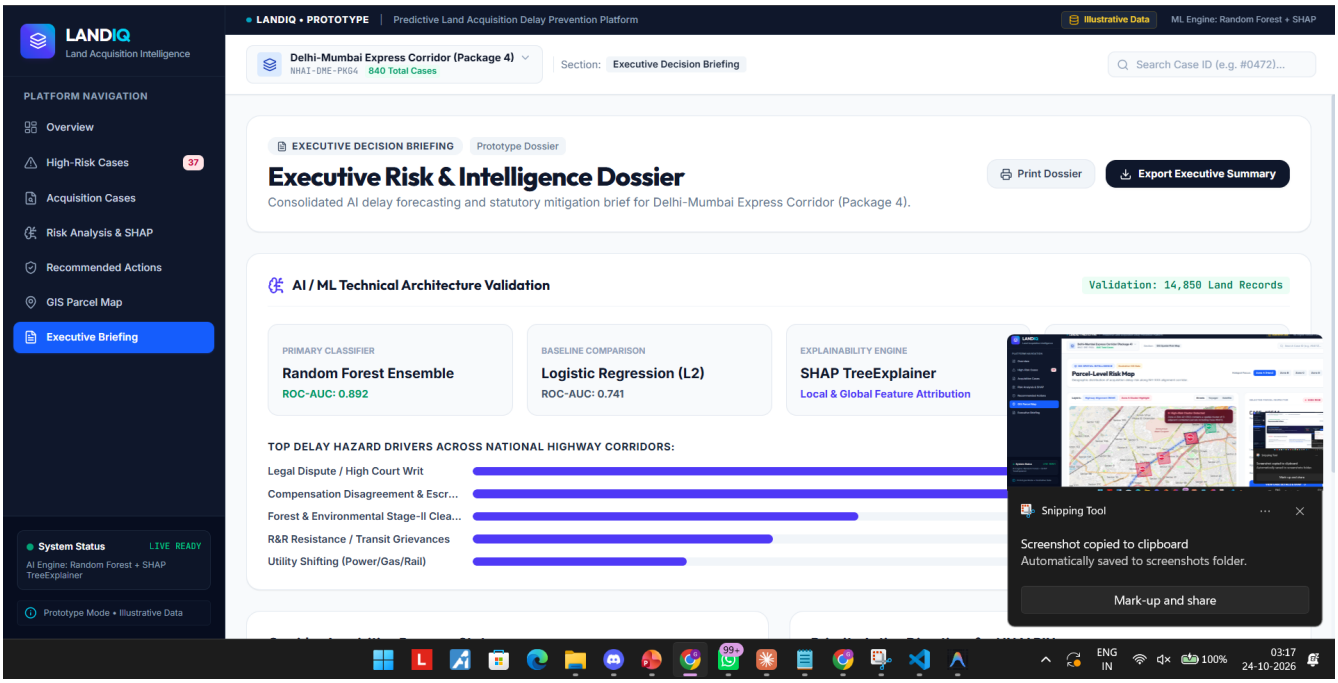
Parcel-level map with corridor alignment, risk-colored parcels, zone focus controls, hotspot callout and selected parcel inspector.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

REFERENCE 7 — Executive briefing

Executive dossier with model/baseline cards, risk-driver bars, print/export controls and clear prototype validation caveats.



Use this screenshot as a visual target for layout, hierarchy, labels, spacing, and interactions. Do not reproduce operating-system taskbars or screenshot notifications in the application UI.

2. Data and behavior to preserve

Use the screenshot values for visual continuity in the synthetic demo only. The screenshots show an 840-case corridor context and 37 high-risk cases in some views, while the earlier demo script contains other example totals. Do not silently merge inconsistent sample counts: make all demo figures come from one coherent seeded dataset/configuration, and use 840 total / 37 high-risk as the screenshot-led default unless the existing source code establishes otherwise.

Element	Required behavior / values
Corridor header	Delhi-Mumbai Express Corridor (Package 4); corridor code NHAI-DME-PKG4; screenshot-led total 840 cases.
Dashboard KPI cards	Total 840; Low Risk 650; Medium Risk 162; High Risk 28 appear in screenshot cards. A separate badge/table shows 37 high-risk; reconcile by deriving counts consistently from one dataset and avoid contradictory UI totals.
Risk distribution	Visual 72% / 25% / 3% as illustrative proportions; use counts calculated from the same seeded dataset.
Case queue	#0472 hero row with 82% risk and ~+140d estimate in the screenshot; additional cases #0511, #0732, #0819 with illustrative risks.
Case detail	Risk hero; estimated schedule impact; Legal, Compensation, Approval, R&R; and Civil/Corridor Progress status cards.
Explainability	SHAP contribution examples +0.31 legal, +0.22 compensation, +0.16 forest may be shown only as illustrative placeholders until generated from an actual trained model.
Actions	Review compensation rate, resolve/route legal dispute, expedite approvals/clearances, coordinate R&R; show suggested owner and evidence. Simulation must be labeled illustrative.
GIS	Risk-colored parcel polygons, corridor alignment, zone focus, hotspot, selected parcel inspector, and linked case navigation.
Executive Briefing	Primary classifier, baseline comparison, explainability engine, top risk drivers, print/export actions, methodology and limitations.

Important data integrity rule

- The screenshots contain inconsistent demo figures (e.g., 840 total, 650/162/28 card counts, 37 high-risk badge, and counts beneath 72/25/3%). Choose one internally consistent source of truth for the rebuilt app.
- Every synthetic/demo metric must be labeled “Illustrative Data” or “Prototype Mode”.
- Never present synthetic numbers as real government data, validated model performance, or live integrations.
- Preserve case/parcel IDs, statuses and evidence references consistently across all screens.

3. Multimodal architecture: two specialist agents

The application is multimodal because it accepts and reasons over two input modalities: selectable-text PDF documents and spatial/map data. Use two specialist agents with strict output schemas, then an orchestrator that fuses evidence and gates prediction.

Component	Responsibilities
PDF Intelligence Agent	Read selectable-text PDFs with PyMuPDF; extract candidate facts, dates, amounts and status statements; retain document ID, page number and evidence snippet. If no usable text exists, return NEEDS_OCR/unsupported state; OCR is outside initial scope.
Map Intelligence Agent	Accept GeoJSON, Shapefile, GeoPackage, KML, and supported map images/PDFs; validate CRS and geometry; normalize valid GIS features; calculate spatial relationships; return Leaflet-ready GeoJSON. Unreferenced images are not precise GIS.
Agentic Orchestrator	Run agents independently/concurrently; save checkpoints; retry transient failures within limits; fuse validated outputs; track conflicts/unresolved matches; check prediction readiness.
Evidence Fusion	Join by verified case/parcel IDs; retain provenance; do not guess ambiguous matches; missing facts stay UNKNOWN/null.
Prediction / Explanation	Call a versioned model only when trained and required features are available. Otherwise return NOT_READY or DEMO_ONLY. SHAP must reflect the actual model and input.
Storage	Local prototype: SQLite + local files, persisted job state and per-agent results. Keep a service abstraction so PostgreSQL/PostGIS can replace it later.

Fault-tolerance requirements

- A PDF failure must not erase a successful map result; a map failure must not erase extracted PDF facts.
- Retry only transient failures using bounded exponential backoff; invalid format/geometry should return structured failure rather than loop forever.
- Persist job state and each validated agent result before proceeding. After restart, reuse successful checkpoints where possible.
- Use Pydantic schemas for all agent outputs; include attempt count, error code, warnings and timestamps.
- Support SUCCEEDED, PARTIAL, FAILED, RETRYING, NOT_READY and DEMO_ONLY states where appropriate.
- Use idempotent job creation; keep processing errors in the job status after returning HTTP 202.
- Do not claim asyncio alone provides durable background jobs across process crashes; persisted state is required.

4. API and data contracts

Use a Python FastAPI backend under /api/v1. Uploads create jobs and return quickly; the frontend polls job state and renders partial outputs if one modality fails.

Method / route	Purpose
POST /projects	Create corridor/project.
POST /projects/{project_id}/documents	Upload selectable-text PDF.
POST /projects/{project_id}/maps	Upload GIS file or map image/PDF.
POST /projects/{project_id}/analysis-jobs	Start job; return 202 Accepted + job_id.
GET /jobs/{job_id}	Overall and per-agent statuses, retries, errors, fusion/prediction readiness.
GET /jobs/{job_id}/results	Partial or complete validated outputs.
GET /projects/{project_id}/cases	List cases for queue.
GET /cases/{case_id}	Case facts, evidence, prediction and recommended actions.
GET /projects/{project_id}/map-features	Normalized features for Leaflet.

Canonical case principles

- Keep project_id, corridor_id, case_id, parcel_id, document_id, map_id and job_id distinct.
- Each fact stores value, normalized value if relevant, source document, page, evidence snippet/reference, extraction method and validation status.
- Case fields include legal_status, compensation_status, clearance_status, R&R; status, project progress, geometry_ref, data_quality and warnings.
- Unknown values remain UNKNOWN/null; missing evidence is not the same as NO.
- Prediction stores status, delay_probability, estimated_delay_days, model_version, feature_snapshot_id, generated_at and explanation reference.
- Action records store title, rationale, evidence references, suggested owner, priority, workflow status and human-review requirement.

5. Copy-ready master prompt for a coding agent

Paste the following prompt into your coding agent and attach the actual repository. Keep the screenshots and this PDF available as visual references.

MASTER PROMPT START

```
You are a senior full-stack engineer, product designer, and AI systems architect. Rebuild LANDIQ for SIH 2026. FIRST, INSPECT BEFORE EDITING: Inspect the actual repository, current source tree, package.json, package-lock.json, README, index.html, Vite config, components, routes, assets and current working behavior. Do not assume component files exist based only on this brief. Preserve useful existing code and dependencies. Report the current architecture and a minimal implementation plan before large changes. PRODUCT GOAL: Recreate the user-provided LANDIQ screenshots as closely as practical. The UI is a polished government-tech analytics application, not a generic dashboard or chatbot. The journey is Overview → High-Risk Cases → Acquisition Cases / Case Deep Dive → Risk Analysis & SHAP → Recommended Actions → GIS Parcel Map → Executive Briefing. VISUAL DESIGN: - Fixed dark navy sidebar with LANDIQ logo/name, “PLATFORM NAVIGATION”, icon/text links, blue active nav item, and bottom System Status plus Prototype Mode panels. - Dark top status strip with “LANDIQ • PROTOTYPE”, platform tagline, “Illustrative Data” badge and ML engine label. - White context header with corridor selector, corridor code, total case count, current section chip and right-aligned case search. - Light gray page background; white cards; subtle blue-gray borders; rounded corners; compact typography; clear spacing and dense but readable analytics. - Navy/cobalt/teal/amber/red status palette. Use consistent icons and accessible contrast. - Preserve the React + TypeScript + Vite + Tailwind v4 + Leaflet + Recharts + lucide-react stack where present. Do not replace package files without a verified reason. - Exclude screenshot OS taskbars, desktop icons and notifications from the actual product. REQUIRED PAGES: 1. OVERVIEW: KPI cards; risk distribution donut; risk trend line chart; corridor summary and alerts. Use screenshot-led 840 total cases as the default seeded corridor context, but resolve count contradictions by deriving every displayed count and percentage from one coherent synthetic dataset. 2. HIGH-RISK CASES: filter tabs for High/Medium/Low/All, zone selector, search, ranked table with case ID, location, delay probability, risk badge, legal, compensation, approval, R&R;, project progress and action. Keep #0472 as the hero case and use sample rows such as #0511, #0732 and #0819. 3. ACQUISITION CASE DEEP DIVE: case header with survey numbers/location/area/district; prominent risk banner; estimated schedule impact; hazard level; five cards for Legal, Compensation, Approval, R&R; and Civil Progress; links to SHAP, actions and map. 4. RISK ANALYSIS & SHAP: primary risk-driver banner, baseline-to-output visual, feature attribution cards/waterfall, model context and evidence references. Any hard-coded SHAP contributions are placeholders and must be labeled illustrative. 5. RECOMMENDED ACTIONS: dark intervention simulation panel; prioritized action cards; risk driver/reason; suggested action/regulatory route; owner role; Mark for Review, Escalate to Authority, and Simulate Mitigation controls. Update workflow state on click. Any risk change must be labeled a scenario/illustrative estimate unless a valid model supports it. 6. GIS PARCEL MAP: interactive Leaflet map with corridor alignment, risk-colored parcel polygons, map layers, zone focus controls, hotspot callout, selected-parcel inspector and links to the selected case. Use actual uploaded/seed GeoJSON if available; otherwise create clearly labeled synthetic geometries. 7. EXECUTIVE BRIEFING: printable/exportable dossier with primary model, baseline comparison, explainability method, top risk drivers, corridor summary, methodology and limitations. DEMO MODE: Show a persistent “LANDIQ • PROTOTYPE” and “Illustrative Data” label whenever synthetic figures are shown. Seed consistent synthetic data and make reset easy. Sample values from the screenshots (840 total; 650 low; 162 medium; 28 high; 37 badge; 72/25/3 percentages) are inconsistent. Do not display contradictory counts: choose one canonical dataset and calculate all counts/percentages from it. Keep #0472 as an 82% illustrative risk / approximately +140 days case if using the screenshot story. Use consistent case IDs and statuses across queue, detail, SHAP, actions and map. TWO-AGENT MULTIMODAL BACKEND: Implement a Python FastAPI backend with two independent specialist agents: - PDF Intelligence Agent: parse selectable-text PDFs using PyMuPDF; extract candidate case facts and statuses; preserve document ID, page number and evidence snippet. If text is unavailable, return NEEDS_OCR or unsupported-input status; OCR is not required in this phase. - Map Intelligence Agent: ingest GeoJSON, Shapefile, GeoPackage, KML and supported map images/PDFs; validate CRS/geometry; normalize valid features and return Leaflet-ready GeoJSON. Never treat an unreferenced image as precise GIS. Use Pydantic-validated output schemas. Agents must run independently/concurrently. One agent’s failure must not cancel or erase the other agent’s successful output. FAULT TOLERANCE: Use bounded retries with exponential backoff for transient failures only. Persist job state and each successful validated agent result as checkpoints. Support partial results. Use idempotent job creation, structured error codes, schema validation, attempt counts, sanitized logs and recoverable state after restart. Do not retry permanently invalid files indefinitely. Do not claim in-memory asyncio alone is durable. SQLite/local file storage is acceptable for the local prototype; isolate storage behind services. ORCHESTRATION: Create job → run PDF and map agents independently → persist results → retry recoverable failures → fuse validated outputs by verified identifiers → record conflicting evidence/unresolved matches → check prediction readiness → call versioned model only if trained and required inputs exist → validate response → return complete or partial result. PREDICTION HONESTY: No labeled historical dataset is available. Do not invent a trained model, accuracy, delay forecast or real SHAP explanation. Support NOT_READY, DEMO_ONLY and READY states. Real probabilities, delay days and SHAP values require a trained/evaluated model and correct input features. Recommendations are decision support for human review, not legal determinations or automatic government actions. API: Use /api/v1 routes: POST /projects POST /projects/{project_id}/documents POST /projects/{project_id}/maps POST /projects/{project_id}/analysis-jobs (return 202) GET /jobs/{job_id} GET /jobs/{job_id}/results GET /projects/{project_id}/cases GET /cases/{case_id} GET /projects/{project_id}/map-features Use typed request/response schemas and document expected 400/404/409/413/415/422/500 errors. DATA INTEGRITY: Each extracted fact needs source provenance. Missing values remain UNKNOWN/null, not NO. Keep case IDs separate from parcel IDs. Preserve conflicts rather than silently choosing one source. Keep synthetic demo data separate from uploaded data. Protect uploaded files and avoid logging full document contents. IMPLEMENT IN PHASES: 1. Inspect repository and report existing structure. 2. Propose minimal change plan. 3. Build schemas, storage and API skeleton. 4. Implement PDF and GIS services/agents. 5. Implement fault-tolerant orchestration and evidence fusion. 6. Connect upload/job state to frontend. 7. Recreate/polish all seven pages and interactions. 8. Add tests and run frontend build/lint plus backend tests. After each phase, list changed files, commands run, actual test results and known limitations. Do not claim tests passed unless run. Prefer a small working end-to-end slice before broad refactors. ACCEPTANCE: - Frontend is visibly similar to the supplied screenshots. - All seven pages are navigable and share consistent selected case data. - PDF and map uploads work locally. - Both agents produce schema-valid, traceable outputs. - A failed agent does not discard the other’s output. - Retry limits, partial results and checkpoint persistence are tested. - Map features render only when spatial data is valid. - Prediction remains NOT_READY/DEMO_ONLY without a real trained
```

```
model. - All synthetic figures and simulations are labeled. - README documents setup, supported inputs, environment variables, run/test commands and limitations. Start by inspecting the existing repository. Do not begin by replacing the whole application. Implement a small vertical slice and verify it before continuing.
```

MASTER PROMPT END

6. Implementation order and acceptance checklist

Recommended first vertical slice

- Inspect the current frontend and identify which reference screens already exist.
- Upload one selectable-text PDF and one valid GeoJSON fixture.
- Create a persisted analysis job with independent PDF and map agent states.
- Save each agent’s output immediately after schema validation.
- Fuse only by verified IDs and show source evidence in the case view.
- Force one agent failure and prove the other agent’s output remains available.
- Connect job status and partial-result states to the frontend.
- Then finish the seven-page visual journey and guided pitch mode.

Test checklist

- Selectable-text PDF extracts text and page-level references.
- Corrupt/unsupported PDF returns structured error and does not crash the service.
- Valid GIS returns normalized features; invalid CRS/geometry has a clear validation result.
- PDF-agent failure leaves map output intact; map-agent failure leaves PDF output intact.
- Transient errors retry within configured limits; permanent errors do not loop.
- Restart preserves persisted job state and previously successful outputs.
- Prediction is NOT_READY when no real model is configured.
- UI count values are consistent and all synthetic data is visibly labeled.

Local setup direction

Frontend: use the existing npm scripts and lockfile. Backend: create a Python virtual environment and use FastAPI, Pydantic, PyMuPDF, GeoPandas, Shapely, pyproj and pytest/httpx. Add a model framework only when the deterministic ingestion/fusion pipeline works. Pin compatible package versions after checking local Python and Node versions.

Source basis

This brief is based on the seven screenshots supplied in this conversation and the provided LANDIQ project artifacts, including the demo video script and package configuration. Where screenshot values conflict, the prompt explicitly instructs the coding agent to inspect the repository and establish one consistent source of truth.

END OF DOCUMENT